

מדריך: בדיקות ביצועים ואוטומציה מקצועית

חלק 1: המדריך של מיקרוסופט לבדיקות ביצועים (PE:06)

חלק 2: בדיקות ביצועים ועומס, JMeter ומדידה נכונה

חלק 3: פרויקטים בקוד פתוח ללמידה של תשתיות בדיקות לחומרה ותקשורת

אוקטובר 2026

תוכן העניינים

חלק 1: המדריך של מיקרוסופט לבדיקות ביצועים (PE:06) 4

4	ההמלצה עצמה
4	המונחים של המדריך
5	אסטרטגיה 1: להגדיר יעדים מדידים
6	אסטרטגיה 2: להתחיל מוקדם ולבדוק ברציפות
6	אסטרטגיה 3: לבדוק בתנאים אמיתיים
7	אסטרטגיה 4: לאמת שינויים בעזרת השערות
7	אסטרטגיה 5: להריץ כמה סוגי בדיקות
8	אסטרטגיה 6: נתונים ודפוסי שימוש אמיתיים
9	אסטרטגיה 7: להשתמש בתוצאות כדי לקבל החלטות
9	הכלים שמיקרוסופט ממליצה עליהם
10	המאמרים הקשורים במדריך
10	איך זה מתורגם לעבודה שלך

חלק 2: בדיקות ביצועים מול בדיקות עומס - איך מגדירים עומס, ומה JMeter באמת

עושה (גם בחומרה) 11

11	תקציר
11	ממצאים מרכזיים
12	פרק 1: האם בדיקות ביצועים "עומדות בפני עצמן"? ההבדל בין ביצועים לעומס
13	פרק 2: הכלים לבדיקות עומס
15	פרק 3: מה זה עומס ואיך מגדירים אותו
17	פרק 4: מה זה JMeter
19	פרק 5: האם JMeter מתאים לבדיקות חומרה?
20	פרק 6: מתודולוגיה מקצועית של בדיקות ביצועים
22	פרק 7: דוגמת קוד - מחולל עומס פתוח ב-Python שמוודד latency נכון
23	המלצות
23	הסתייגויות
24	מקורות

חלק 3: איפה לראות פרויקט בדיקות ואוטומציה מקצועי אמיתי 25

25	3.1 תקציר
25	3.2 תשתיות לבדיקות חומרה, embedded ומערכת

29	3.3 רשתות, פרוטוקולים ותעבורה
31	3.4 פרויקטים עם תיקיית בדיקות לדוגמה
31	3.5 תבניות ורשימות
32	3.6 איך נראה פרויקט אוטומציה מקצועי
34	3.7 סדר קריאה ל-4 עד 6 שבועות
35	3.8 הסתייגויות

חלק 1: המדריך של מיקרוסופט לבדיקות ביצועים

(PE:06)

השורה התחתונה: המדריך הרשמי של מיקרוסופט לבדיקות ביצועים הוא Architecture strategies for performance testing, והוא חלק מ-Azure Well-Architected Framework. הוא מבוסס על פרקטיקות הבדיקה הבסיסיות שמתוארות במאמר OE:09 על אסטרטגיות בדיקה, ומיקרוסופט ממליצה לקרוא את המאמר ההוא קודם. המדריך בנוי משבע אסטרטגיות: מגדירים יעדים מדידים, מתחילים מוקדם ובודקים ברציפות, בודקים בתנאים אמיתיים, מאמתים שינויים בעזרת השערות, מריצים כמה סוגי בדיקות, משתמשים בנתונים ריאליים, ומשתמשים בתוצאות כדי לקבל החלטות. בזמן האחרון המדריך עבר עדכון מקיף כדי ליישר אותו עם מדריך הבדיקות המעודכן, והגרסה המתוארת כאן היא מאוגוסט 2026.

מקור: [Architecture strategies for performance testing – Microsoft Learn](#)

ההמלצה עצמה

המלצת ביצועים 6 - PE:06

ההמלצה היא לשפר את הביצועים של המערכת בעזרת בדיקות קבועות בסביבה שדומה לייצור, כדי לוודא שהמערכת מגיעה ליעדי הביצועים ולמטרות העסקיות.

איך מיקרוסופט מגדירה בדיקות ביצועים

בדיקות ביצועים הן פרקטיקה לא-פונקציונלית שבודקת איך המערכת מתנהגת בתנאים שונים. הן עוזרות לזהות הידרדרות מוקדם, לטפל בבעיות לפני שהן מתפוצצות, ולשמור על עמידה בהסכמי רמת השירות. כשמודדים זמני תגובה, תפוקה, שימוש במשאבים ויציבות, אוספים ראיות לכך שהמערכת עומדת ביעדים באופן עקבי.

המונחים של המדריך

מיקרוסופט פותחת את המדריך בטבלת מונחים. כדאי להכיר אותם, כי כל שאר המדריך נשען עליהם:

יעדי ביצועים - Performance Targets

הערכים הספציפיים שהמערכת חייבת לעמוד בהם, כמו זמן תגובה, תפוקה או מספר משתמשים בו-זמנית.

ספי ביצועים - Performance Thresholds

הגבולות שמפרידים בין ביצועים מקובלים לבלתי מקובלים במדד מסוים.

תקציב ביצועים - Performance Budget

החלק מיעד הביצועים הכולל שמוקצה לכל שכבה או רכיב במערכת.

תקציב שגיאות - Error Budget

רמת השגיאות או הכשלים המותרת, שנגזרת מיעדי רמת השירות (SLO).

קריטריוני קבלה - Acceptance Criteria

התנאים שתוצאת בדיקה חייבת לעמוד בהם כדי שהמערכת תעבור את דרישות הביצועים.

ניסוי מבוסס השערה - Hypothesis-Driven Experimentation

שיטה שבה מנסחים תחזית לגבי ביצועים, בודקים אותה מול קו בסיס ומאמתים אותה בתוצאות מדודות.

קו בסיס - Performance Baseline

סט מדדים שמייצג את התנהגות המערכת בתנאים רגילים, כפי שאומתה בבדיקות.

טרנזקציות סינתטיות - Synthetic Transactions

בקשות מתוסרטות שמדמות פעולות של משתמשים אמיתיים, כדי למדוד ביצועים בתנאים מבוקרים.

רגרסיה בביצועים - Performance Regression

ירידה בביצועים ביחס לקו הבסיס, שנגרמה משינוי בקוד, בהגדרות או בתשתית.

סחיפה בביצועים - Performance Drift

ירידה הדרגתית בביצועים לאורך זמן, שלא שמים לב אליה בלי בדיקות קבועות מול קו הבסיס.

ההבדל בין שני המונחים האחרונים: רגרסיה היא קפיצה בגלל שינוי מסוים. סחיפה היא שחיקה איטית שאף שינוי בודד לא אחראי לה.

אסטרטגיה 1: להגדיר יעדים מדידים

הרעיון: יעדים מדידים הופכים ציפיות סובייקטיביות לקריטריונים אובייקטיביים שאפשר לבדוק.

שלב א: להגדיר יעדים ולחלק תקציבים

מגדירים ומתעדים יעדים ספציפיים, למשל כמה משתמשים בו-זמנית צריך לתמוך, מוודאים שהם תואמים ל-SLO, ומתרגמים אותם ליעדי בדיקה מדידים.

למה צריך תקציב לכל שכבה? כשבדיקה נכשלת, התקציבים עוזרים לזהות איזו שכבה אחראית. בלי תקציבים, בדיקה שנכשלה אומרת רק שהיעדים לא הושגו, ולא איפה הבעיה.

הדוגמה של מיקרוסופט: תקציב של 400 מילישניות לזמן תגובה של ה-API, 150 מילישניות לשאילתות מסד נתונים, ותקרה של 1% לבקשות שנכשלו. כשבדיקה נכשלת, משווים כל שכבה לתקציב שלה ומבינים אם הבעיה היא API איטי, שאילתות איטיות או קפיצה בשגיאות.

אזהרה חשובה: לא מגדירים SLO לפני שמבינים את זרימות המשתמש ואת דרישות הביצועים. יעדים צריכים להתבסס על צרכים אמיתיים ומטרות עסקיות, לא על מספרים שרירותיים.

שלב ב: להגדיר קריטריוני קבלה עם סף ברור

מבססים את הקריטריונים על מדדים כמו השהיה, זמני תגובה, תפוקה, ניצול משאבים ושיעור שגיאות. לכל מדד קובעים סף, כך שהבדיקה מחזירה תשובה חד-משמעית של עבר או נכשל. אם ה-SLO דורש ש-95% מהבקשות יסתיימו תוך 200 מילישניות, הסף הוא 200 מילישניות באחוזון ה-95, וכל ריצה שחורגת ממנו נכשלת.

אסטרטגיה 2: להתחיל מוקדם ולבדוק ברציפות

הרעיון: ניתוח ביצועים מוקדם תופס צווארי בקבוק בארכיטקטורה לפני שהתיקון שלהם נעשה יקר.

להתחיל מוקדם

לא צריך אפליקציה שלמה כדי להתחיל. מפתחים יכולים לעשות פרופילינג לקוד על המחשב שלהם, למדוד זמני תגובה ולזהות פעולות שצורכות הרבה משאבים. בדיקה מוקדמת משפיעה על החלטות תכנון ומאמתת בחירות ארכיטקטוניות מול היעדים.

לבדוק ברציפות

כל שינוי בקוד עלול להכניס רגרסיה בביצועים. לכן משלבים בדיקות ביצועים ב-pipeline של הפריסה, ומריצים בדיקות אוטומטיות תקופתיות כדי לזהות סחיפה לפני שהיא מגיעה לייצור.

המחיר: בדיקות מוקדמות דורשות תשתית ייעודית ומומחיות, ולכן הן מעלות עלויות. צריך לשקול את ההשקעה מול המחיר של בעיות ביצועים שמתגלות מאוחר ושל תקלות בייצור.

אסטרטגיה 3: לבדוק בתנאים אמיתיים

הרעיון: הבדיקות צריכות להתאים לתנאים בעולם האמיתי, אחרת התוצאות לא אומרות הרבה.

סביבת בדיקה שמשקפת את הייצור

סביבת הבדיקה צריכה להיות דומה לייצור ככל שאפשר, ורמת הדמיון נקבעת לפי פרופיל הסיכון של המערכת.

במערכות קריטיות מתאימים את הסביבה לייצור בדיוק, בכל אלה: סוגי המחשוב וההגדרות שלהם, הגדרות הרחבה אוטומטית, הגדרות מטמון, תנאי רשת (השהיה ורוחב פס), ותלויות חיצוניות.

במערכות לא קריטיות אפשר לבדוק בסביבה מוקטנת שמחקה את הייצור, ולקבל תובנות שימושיות בעלות נמוכה יותר.

בדיקה בייצור עצמו

למה: סביבות בדיקה לא יכולות לשחזר את כל התנאים האמיתיים. חלק מהדרישות אפשר לאמת רק במקום שבו משתמשים, נתונים ותשתית אמיתיים נפגשים.

מה בדיקה בייצור חושפת: דפוסי התנהגות ונפחי נתונים אמיתיים, השהיה ורוחב פס אמיתיים, השפעות של פיזור גאוגרפי, ביצועים של ממשקים ושירותים של צד שלישי, והתנהגות אמיתית של מטמון ותשתית.

איך עושים את זה בזהירות:

- מתזמנים את הבדיקות לשעות שאינן שעות שיא.
- משתמשים בגישה הדרגתית: מתחילים מאחוז קטן של תעבורה ומגדילים בהדרגה, תוך ניטור של זמני תגובה, תפוקה, שגיאות וניצול משאבים בכל שלב. כך מגבילים את הסיכון ועדיין מוצאים את נקודת השבירה.
- מקצים קיבולת נוספת, כדי לספוג את העומס שהבדיקות עצמן מייצרות.

הסיכון: בדיקה בייצור משפיעה ישירות על לקוחות אמיתיים, כי היא מוסיפה עומס ועלולה לשבש תעבורה. צריך אמצעי הגנה, חשיפה מוגבלת ותוכנית חזרה לאחור.

אסטרטגיה 4: לאמת שינויים בעזרת השערות

הרעיון: מתחילים מהשערה ממוקדת לגבי ביצועי המערכת, ומגדירים קריטריוני הצלחה מדידים שמובילים להחלטות.

הדוגמה של מיקרוסופט:

ההשערה היא שהוספת אינדקס לטבלת ההזמנות תקצר את זמן השאילתה ב-70% בעומס שיא. קו הבסיס הוא הסכמה הנוכחית, והגרסה הנבדקת היא הסכמה עם האינדקס. מריצים את אותה בדיקת עומס על שתי הגרסאות, אוספים השהיית שאילתות, צריכת מעבד של מסד הנתונים ותפוקה, ומשווים.

המחיר: צריך להריץ את אותן בדיקות פעמיים, על הבסיס ועל הגרסה החדשה, וזה מייקר את התשתית ואת זמן ההרצה. לכן מתמקדים בשינויים עם השפעה גבוהה.

המבנה הוא בעצם ניסוי מדעי: השערה, קבוצת ביקורת (קו הבסיס), קבוצת ניסוי (הגרסה החדשה), ומדידה זהה בשתייהן.

אסטרטגיה 5: להריץ כמה סוגי בדיקות

הרעיון: כל סוג בדיקה מכוון להיבט אחר של הביצועים. למשל, בדיקת מאמץ מוצאת את נקודת השבירה בעומס שיא, אבל רק בדיקת סיבולת חושפת דליפות זיכרון שמופיעות אחרי שעות או ימים.

סוג	מטרה	מתי	מה חושפת	סביבה
עומס	לזוודא שהמערכת מחזיקה את נפח המשתמשים הצפוי בשימוש רגיל ובשיא	מוקדם ולעיתים קרובות	קו בסיס, גבולות קיבולת, יעילות ההרחבה	סביבה דומה לייצור
מאמץ	להבין את הגבולות ואת נקודות השבירה	לפני שהמערכת מוכנה לייצור	קיבולת מקסימלית, אופני כישלון, התאוששות	סביבה ייעודית לבדיקות ביצועים
קפיצה	לזוודא שהמערכת מתמודדת עם קפיצות פתאומיות בתעבורה	מוקדם, במיוחד במערכות חשופות לציבור	מהירות ההרחבה, טיפול בתורים, ירידה מבוקרת	סביבת staging או ייצור
סיבולת	לגלות בעיות שמופיעות רק לאורך זמן	אחרי שבדיקות העומס הראשונות עברו	דליפות זיכרון, מיצוי משאבים, בעיות במאגרי חיבורים	סביבה דומה לייצור עם כל המשאבים

ההמלצה על הסדר: לא מנסים להפעיל את כל הסוגים בבת אחת. מתחילים מבדיקת עומס בסיסית כדי להבין את קו הבסיס, ומתרחבים לבדיקות מאמץ, קפיצה ובסוף סיבולת ככל שמזהים סיכונים וצוברים ניסיון.

המחיר: כל הסוגים ביחד דורשים הרבה זמן ותשתית, ולכן מתאימים את ההשקעה לסיכון העסקי.

במדריך נוסף של מיקרוסופט על בדיקות, שמתמקד בבחירת כלים, מופיעים גם בדיקות סיבולת תחת עומס גבוה ומתמשך לזיהוי הידרדרות ודליפות, ובדיקות תאימות שבדקות ביצועים על פלטפורמות ומכשירים שונים.

אסטרטגיה 6: נתונים ודפוסי שימוש אמיתיים

הרעיון: בדיקה עם נתונים ריאליים נותנת תמונה מדויקת של צריכת משאבים, של התנהגות המערכת ושל בעיות ביצועים נסתרות.

נתונים מגוונים

בונים סטים של נתונים שמייצגים תרחישים, פרופילי משתמשים ונפחים שונים, ומשתמשים בווריאציות ובאקראיות כדי לחקות משתמשים אמיתיים. כוללים גם מקרי קצה שעלולים לגרום לבעיות, כמו payloads גדולים, שאילתות מורכבות ומקביליות גבוהה.

טרנזקציות סינתטיות

מתסרטים טרנזקציות שמחקות זרימות עבודה של משתמשים אמיתיים, ומריצים אותן שוב ושוב כדי לייצר עומס שמשקף את השימוש בפועל.

תרחישים לפי שימוש אמיתי

התרחישים צריכים לשקף גישה מקבילה של משתמשים, תקופות שיא ורצפי פעולות ספציפיים.

המלצה שמפתיעה רבים: לא להחליף תלויות חיצוניות ברכיבים מדומים

בבדיקה תחת עומס כוללים קריאות אמיתיות לממשקים של צד שלישי. רכיבים מדומים מריצים את הבדיקות מהר ובצורה צפויה, אבל מסתירים בעיות ביצועים אמיתיות.

שימי לב: ההמלצה הזו נכונה לבדיקות ביצועים. בבדיקות יחידה ובבדיקות פונקציונליות רכיבים מדומים עדיין מצוינים.

אסטרטגיה 7: להשתמש בתוצאות כדי לקבל החלטות

לקבוע קו בסיס

רושמים את מדדי הביצועים בבדיקות הראשונות. זה קו הבסיס, תמונת מצב של ביצועים "רגילים". בכל ריצה הבאה משווים אליו כדי לזהות שינויים.

לתעדף לפי נתונים

כשבוחנים את הנתונים, מביאים בחשבון את ההשפעה על המשתמשים, התדירות, עלות התיקון והסיכון שבשינוי. מחפשים דפוסים שמראים איפה הביצועים יורדים, ומשתמשים בזה כדי לתעדף.

אופטימיזציה איטרטיבית

מקצים זמן ייעודי לאופטימיזציה במחזור הפיתוח, ומשתמשים בקו הבסיס כדי לוודא שכל שינוי משפר בלי להכניס גרסיות.

לקשר לתוצאות עסקיות

מקשרים שיפורי ביצועים לתוצאות כמו הכנסות, מעורבות משתמשים ושביעות רצון, כדי להצדיק המשך השקעה.

לעדכן את קו הבסיס

אחרי שינויים משמעותיים, כמו שינוי ארכיטקטורה, פיצ'רים חדשים או שינויי הרחבה, מעדכנים את קו הבסיס כדי שהיעדים יישארו רלוונטיים.

הכלים שמיקרוסופט ממליצה עליהם

צינור אינטגרציה - [Azure Pipelines](#)

מאפשר לשלב בדיקות ביצועים ב-CD/CI, ולהוסיף בדיקת עומס כשלב ב-pipeline.

הזרקת תקלות - [Azure Chaos Studio](#)

מזריק תקלות אמיתיות לאפליקציה, כדי להריץ ניסויים מבוקרים ולמדוד את עמידות המערכת.

שירות בדיקות עומס - [Azure Load Testing](#)

מייצר עומס בקנה מידה גדול. אפשר להגדיר בו קריטריונים כמו זמן תגובה ממוצע או סף שגיאות, לעצור בדיקה אוטומטית בתנאי שגיאה מסוימים, לעקוב אחרי מדדים בזמן אמת ולהשוות בין ריצות כדי לזהות גרסיות.

ניטור - [Azure Monitor](#) ו-[Application Insights](#)

פתרון לאיסוף וניתוח טלמטריה מהענן ומסביבות מקומיות. Application Insights מוסיף יכולות ניטור אפליקציות, ומשמש גם בפיתוח ובבדיקות וגם בייצור.

המאמרים הקשורים במדריך

המדריך הוא חלק ממשפחה של מאמרים. לפי הסדר שבו כדאי לקרוא:

1. [OE:09 - אסטרטגיות בדיקה כלליות](#): הבסיס שמיקרוסופט ממליצה לקרוא לפני מדריך הביצועים.
2. [PE:06 - בדיקות ביצועים](#): המדריך שהוסבר כאן.
3. [ניטור ביצועים](#): מאמר שמתאר מדידת ביצועים בכמה שכבות, שאוספת השהיה, תפוקה והתנהגות משאבים כדי לבנות קו בסיס ולזהות הידרדרות. אחת ההמלצות שם רלוונטית במיוחד לבדיקות: לא לערבב נתונים מסביבות בדיקה עם נתוני ייצור, כי זה מעוות את התוצאות ומסתיר בעיות אמיתיות.
4. [מאמרים משיקים](#): המדריך מפנה גם להמלצות על [בדיקות אבטחה](#) ועל [אסטרטגיית בדיקות אמינות](#).
5. [המושגים של Azure Load Testing](#): משתמשים וירטואליים, בקשות בשנייה וזמן עלייה.

איך זה מתורגם לעבודה שלך

המדריך נכתב לשירותי ענן, אבל כמעט כל אסטרטגיה עובדת גם על מערכות תקשורת וחומרה:

אסטרטגיה	במערכת קשר
יעדים ותקציבים לכל שכבה	עיכוב מקצה לקצה מחולק לקידוד, ערוץ ומסרים. כשבדיקה נכשלת, יודעים איפה לחפש
קריטריוני קבלה באחוזונים	"עיכוב קולי עד 300 מילישניות באחוזון ה-95", ולא "העיכוב נמוך"
להתחיל מוקדם	פרופייילינג של הקודק על מחשב פיתוח, עוד לפני שיש מכשיר
סביבה דומה לייצור	אותה חומרה, אותה קושחה, אותו ערוץ או סימולטור ערוץ מכויל
השערות	"הגדלת חוצץ הג'יטר תוריד את הקטיעות ב-50% במחיר של 20 מילישניות עיכוב"
סוגי בדיקות לפי הסדר	קודם עומס של 30 מכשירים, אחר כך מאמץ, קפיצה, ובסוף 72 שעות סיבולת
נתונים ריאליים	תמהיל אמיתי של קול, טקסט ומיקום, והקלטות קול עם רעש רקע
קו בסיס ורגרסיה	כל גרסת קושחה חדשה נמדדת מול קו הבסיס של הגרסה הקודמת

חלק 2: בדיקות ביצועים מול בדיקות עומס - איך מגדירים עומס, ומה JMeter באמת עושה (גם בחומרה)

בקצרה: בדיקות ביצועים הן לא קטגוריה נפרדת ומקבילה לבדיקות עומס. ב-ISTQB, בעיקר בסילבוס CT-PT, הן "מונח מטרייה". בדיקת עומס היא סוג אחד בתוכן, לצד בדיקות מאמץ, קפיצות, סיבולת, מדרגיות, קיבולת וכו'. בפועל אנשים משתמשים בשני המונחים כמעט כמילים נרדפות.

הקשר לחלק 1: כל ההמלצות של מיקרוסופט שמוזכרות בחלק הזה (תקציבים לכל שכבה, סף באחוזונים, קו בסיס, השערות, סביבה דומה לייצור, נתונים ריאליים וסוגי הבדיקות) מוסברות במלואן בחלק 1, לפי המדריך המקורי.

תקציר

- **יש הבדל, והוא היררכי.** "בדיקות ביצועים" זו המטרייה: כל בדיקה שמודדת זמני תגובה, תפוקה וניצול משאבים. "בדיקת עומס" היא בדיקה אחת מתחתיה, שבודקת איך המערכת מתנהגת בעומס צפוי וריאלי. גם מיקרוסופט ב-Azure Well-Architected מסדרת את זה ככה.
- **עומס זה לא "כמה משתמשים".** עומס זה קצב הגעה של עבודה (בקשות, הודעות, פריימים לשנייה), ביחד עם תמהיל פעולות ופרופיל זמן. אם מודדים latency, צריך מודל פתוח (open model). אחרת הכלי "מסתיר" את הזנב האיטי: זו ה-coordinated omission של Gil Tene תיאר.
- **JMeter** הוא כלי ג'אווה בקוד פתוח שמייצר עומס ברמת הפרוטוקול: HTTP, TCP, JDBC, JMS ועוד. על חומרה הוא בודק רק את הממשק שהיא חושפת ברשת, לא את החומרה עצמה. במערכות משובצות, רדיו ותקשורת עובדים בדרך כלל עם מחוללי תעבורה (iperf3, TRex, Keysight/Ixia, Spirent), עם HIL ועם רתמות Python/C ייעודיות. המדדים שם הם latency, jitter, איבוד חבילות ודטרמיניזם של תזמון.

ממצאים מרכזיים

- ב-ISTQB, סילבוס CT-PT (2018): "Performance testing is an umbrella term including any kind of testing focused on performance (responsiveness) of the system or component under different volumes of load". בדיקת עומס מופיעה שם כאחד מתוך שמונה סוגים.
- גם במילון המונחים של ISTQB בדיקת עומס היא "A type of performance testing". היא מוגדרת כבדיקה של התנהגות המערכת "with increasing load, e.g., numbers of parallel users and/or numbers of transactions".

- ב-ISO/IEC 25010:2023 אין "בדיקות ביצועים" בכלל. יש מאפיין איכות בשם performance efficiency, עם שלושה תת-מאפיינים: time behaviour, resource utilization, capacity. בדיקות ביצועים הן הדרך לאמת את המאפיין הזה.
- מיקרוסופט מגדירה בדיקות ביצועים כ-"a nonfunctional testing practice used to evaluate how a workload behaves under various conditions". בטבלת סוגי הבדיקות שלה מופיעים load, stress, spike ו-soak. ההמלצה שלה: להתחיל מבדיקת עומס בסיסית ולהתרחב משם.
- ב-(Grafana) k6 מסבירים את זה ישר: במודל סגור, כשהמערכת מאטה, קצב הבקשות יורד מעצמו. "In some testing literature, this problem is known as coordinated omission". הפתרון שלהם הוא executors מסוג arrival-rate.
- לפי האתר הרשמי של JMeter, הכלי "works at protocol level" ו-"does not execute the javascript found in HTML pages". כלומר הוא לא דפדפן, וודאי שלא בודק חומרה.

פרק 1: האם בדיקות ביצועים "עומדות בפני עצמן"? ההבדל בין ביצועים לעומס

מה אומרים התקנים

תקן איכות - ISO/IEC 25010

התקן לא מדבר על בדיקות אלא על מאפייני איכות. מאפיין אחד הוא performance efficiency, שמוגדר בגרסת 2023 כ-"capability of a product to perform its functions within specified time and throughput parameters and be efficient in the use of resources under specified conditions".

יש לו שלושה תת-מאפיינים:

- **התנהגות בזמן (time behaviour):** זמני תגובה וקצבי תפוקה שעומדים בדרישות.
- **ניצול משאבים (resource utilization):** לא לצרוך יותר משאבים מהמותר.
- **קיבולת (capacity):** גבולות מקסימליים של פרמטרים, למשל מספר משתמשים בו-זמנית, רוחב פס, תפוקת טרנזקציות וגודל מסד נתונים.

שימי לב שיש שתי גרסאות. בגרסת 2011 ההגדרה הייתה "performance relative to the amount of resources used under stated conditions", והרבה מקורות משניים עדיין מצטטים אותה.

סילבוס ISTQB לבדיקות ביצועים - ISTQB CT-PT

לפי הסילבוס, בדיקות ביצועים הן מטרייה, ומתחתיה שמונה סוגים:

- **בדיקת ביצועים (performance):** זמני תגובה וניצול משאבים, גם בלי עומס כבד. זה הסוג הבסיסי, והוא אחד משמונת הסוגים.
- **בדיקת עומס (load):** רמות עולות של עומס צפוי וריאלי.
- **בדיקת מאמץ (stress):** עומס בגבול הצפוי או מעבר לו, או עבודה עם פחות משאבים מהרגיל.
- **בדיקת מדרגיות (scalability):** עמידה בדרישות עתידיות, למשל יותר משתמשים או יותר נתונים.
- **בדיקת קפיצה (spike):** פרצי עומס פתאומיים וחזרה למצב יציב.

- **בדיקת סיבולת (endurance/soak):** יציבות לאורך זמן, כמו דליפות זיכרון, מאגרי חיבורים ו-thread pools.
- **בדיקת בו-זמניות (concurrency):** פעולות שקורות בדיוק באותו רגע, למשל התחברות המונית.
- **בדיקת קיבולת (capacity):** כמה משתמשים או טרנזקציות המערכת מחזיקה ועדיין עומדת ביעדים.

איפה המונחים מתערבבים בפועל

- בתעשייה אומרים "load test" כמעט על כל בדיקה שמייצרת תעבורה. גם הכלים נקראים "load testing tools", אף שהם מריצים גם stress ו-Azure Load Testing ו-soak. Distributed Load Testing on AWS הם דוגמאות טובות.
- מיקרוסופט משתמשת ב-"performance testing" כשם הקטגוריה (בהמלצה PE:06), ובטבלה שלה load testing הוא סוג אחד. זה תואם את ISTQB.
- אצל ISTQB עצמם יש אי-עקביות קטנה. בגרסאות ישנות של המילון בדיקת ביצועים הייתה "Testing to determine the performance of a software product". בגרסאות מאוחרות יותר היא הוגדרה כבדיקה לקביעת performance efficiency. הכיוון זהה, והניסוח השתנה.
- יש גם מחלוקת אמיתית, והיא על גבול העומס. חלק מהמקורות אומרים ש-load זה "עומס צפוי בלבד" וכל מה שמעבר הוא stress. המילון של ISTQB מדבר על "increasing load ... to determine what load can be handled", וזה כבר נוגע בחיפוש נקודת השבירה. המסקנה המעשית: כשמישהו אומר לך "בדיקת עומס", תשאל מה המטרה. האם לאמת עומס צפוי, או למצוא את נקודת השבירה?

המסקנה שלי

- אם שואלים "יש בדיקות ביצועים בלי עומס?", התשובה היא כן. דוגמאות: מדידת latency של פקודה בודדת, פרופילינג של פונקציה, זמן אתחול של מכשיר, זמן מעבר בין מצבים ברדיו. אלה בדיקות ביצועים לכל דבר, ואין בהן עומס בכלל.
- אם שואלים "יש בדיקת עומס שהיא לא בדיקת ביצועים?", התשובה היא לא. עומס הוא תמיד אמצעי למדוד ביצועים תחת תנאי מסוים.

פרק 2: הכלים לבדיקות עומס

המלצה כללית: בחרים כלי לפי הפרוטוקול, לפי השפה שהצוות שולט בה, ולפי מודל העומס שצריך (פתוח או סגור). הפופולריות פחות חשובה. בשבילך, עם רקע ב-Python, Locust הוא נקודת כניסה טבעית, ו-JMeter שווה ללמוד כי הוא הסטנדרט הוותיק.

כלים בקוד פתוח

כלי ותיק של אפאצ'י - Apache JMeter

- כתוב ב-Java. יש לו GUI לבניית תוכניות בדיקה (קובצי JMX ב-XML), וסקריפטים ב-Groovy דרך JSR223.
- **פרוטוקולים:** HTTP/HTTPS, SOAP/REST, FTP, JDBC, LDAP, JMS, SMTP/POP3/IMAP, TCP, Java objects. פקודות מערכת ו-Java objects.

- **חוזקות:** מכסה הכי הרבה פרוטוקולים מהקופסה, קהילה ענקית, ותמיכה של Azure Load Testing ו-AWS DLT.
- **חולשות:** צורך הרבה זיכרון, קבצי XML לא נוחים ל-code review, ומודל ברירת המחדל הוא סגור (threads).

כלי Python - Locust

- הבדיקות נכתבות "in plain Python code", ואפשר להפיץ אותן על כמה מכונות.
- נבנה בעיקר ל-HTTP, אבל לפי [התיעוד הרשמי](#) אפשר להרחיב אותו לכל מערכת בקשה-תגובה בעזרת custom client. בתיעוד יש דוגמאות ל-XML-RPC ול-gRPC, ויש גם MqttUser מובנה ב-contrib.
- **חוזקות:** Python רגיל, git, ספריות קיימות. מתאים מאוד לרתמות בדיקה של מערכות משובצות.
- **חולשות:** לפי התיעוד של Locust (גרסה 2.46.6), תהליך יחיד שמוגבל לליבה אחת מגיע במקרה הטוב לכ-16,000 בקשות לשנייה עם FastHttpUser ולכ-4,000 עם HttpUser, ולכן בעומס גבוה צריך להריץ כמה תהליכים. בנוסף, ספריות הפרוטוקול צריכות להתאים ל-gevent.

כלי של Grafana - k6

- כתוב ב-Go, והסקריפטים ב-JavaScript.
- **חוזקות:** executors מפורשים למודל פתוח (constant-arrival-rate, ramping-arrival-rate) ולמודל סגור, thresholds שמכשילים pipeline, ואינטגרציה עם Grafana.
- **חולשות:** מתמקד ב-HTTP, WebSocket ו-gRPC. לפרוטוקולים אחרים צריך הרחבות.

כלי מבוסס Scala - Gatling

- DSL ב-Scala, Java או Kotlin, ודוחות HTML חזקים.

כלי Node.js - Artillery

- תרחישים ב-YAML או JavaScript, נוח לבדיקות API ו-WebSocket.

כלים קלים לשורת פקודה - Vegeta, wrk, wrk2

- Vegeta כתוב ב-Go ועובד בקצב קבוע (מודל פתוח).
- wrk הוא כלי HTTP מהיר מאוד ב-C.
- wrk2 הוא הגרסה של Gil Tene ל-wrk. הוא עובד בקצב קבוע ורושם latency עם HdrHistogram, כדי לא ליפול ל-coordinated omission.

שירותי ענן מנהלים

שירות העומס של מיקרוסופט - Azure Load Testing

- מריץ סקריפטים של JMeter או של Locust בלבד: "Azure Load Testing currently does not support other testing frameworks than Apache JMeter and Locust".
- תומך בכל פרוטוקול ש-JMeter תומך בו, כולל מסדי נתונים ותורי הודעות, ואפשר לחבר אותו ל-VNET כדי לבדוק endpoints פרטיים או on-premises.

- לפי מיקרוסופט, מכונות הבדיקה הן JMeter 5.6.3 עם Standard_D4d_v4 ו-Locust 2.33.2. אפשר להגיע עד 400 מנועים ו-100,000 משתמשים וירטואליים בריצה אחת.

Distributed Load Testing on AWS - פתרון העומס המבוזר של אמזון

- פתרון בקוד פתוח שמריץ קונטיינרים על ECS Fargate עם Taurus, ותומך ב-JMeter ו-Locust.
- תומך בריצה מכמה regions, בתזמון, ובדוחות עם p95, p90, p50 ו-p99.

כלים מסחריים

כלי ארגוני ותיק - OpenText LoadRunner (לשעבר Micro Focus/HP)

- לפי OpenText הוא תומך ביותר מ-180 פרוטוקולים וטכנולוגיות, כולל אפליקציות ווב, מובייל, packaged ו-legacy. גרסת Professional שלו נקראת היום OpenText Professional Performance Engineering. יקר, ומקובל בארגונים גדולים.

כלי ארגוני - Tricentis NeoLoad

- עיצוב בדיקות בעזרת GUI ו-as-code, ואינטגרציה עם APM.

פלטפורמת ענן - BlazeMeter

- מריצה סקריפטים של Locust, Gatling, JMeter ו-k6 בענן, עם דשבורדים.

כלים לרשת ולחומרה (פירוט בפרק 5)

- מחוללי תעבורה (Spirent, Keysight/Ixia, TReX, iperf3): בודקים רשת, מכשירי תקשורת ו-DUT ברמת החבילה. הם לא בודקים משתמשים.

פרק 3: מה זה עומס ואיך מגדירים אותו

ההגדרה

פרופיל עומס - load profile

- לפי ISTQB, פרופיל עומס מגדיר כמה משתמשים וירטואליים מבצעים איזה סט טרנזקציות ובאיזה פרק זמן, כפי שהמערכת עשויה לחוות בייצור. הוא נבנה על בסיס פרופיל שימוש.

פרופיל שימוש - operational profile

- הייצוג של סט המשימות שהמערכת מבצעת ושל ההסתברות של כל אחת מהן. במילים פשוטות: תמהיל הפעולות.

כלומר עומס הוא לא מספר אחד. הוא מורכב מארבעה דברים:

- **עוצמה:** כמה עבודה נכנסת ביחידת זמן.
- **תמהיל:** אילו פעולות, ובאיזה יחס ביניהן.

- **צורה בזמן:** עלייה הדרגתית, מצב יציב, שיא וקפיצות.
- **נתונים:** גודל payload, גודל DB, מגוון קלטים. מיקרוסופט ממליצה במפורש על דאטה מגוון ו-edge cases כמו payloads גדולים (חלק 1, אסטרטגיה 6).

משתמשים בו-זמנית מול בקשות לשנייה מול קצב הגעה

משתמשים בו-זמנית - concurrent users

כמה "ישויות" פעילות באותו רגע. מספר זה לבדו לא אומר כמה עבודה המערכת מקבלת.

תפוקה - throughput / RPS

כמה בקשות או הודעות מעובדות בשנייה.

קצב הגעה - arrival rate

כמה עבודה חדשה נכנסת בשנייה, בלי קשר לזמן שלוקח לסיים את הקודמת.

זמן חשיבה - think time

ההשהיה בין פעולות של משתמש אמיתי. ב-JMeter זה נעשה עם Timers.

קצב קבוע לאיטרציה - pacing

קביעת זמן קבוע לכל איטרציה של משתמש, כדי לשלוט בקצב.

הקשר בין המושגים מגיע מתורת התורים (חוק ליטל):

$$\text{משתמשים בו-זמנית} = \text{תפוקה} \times (\text{זמן תגובה} + \text{זמן חשיבה})$$

דוגמה: 50 בקשות לשנייה, זמן תגובה של 0.2 שניות וזמן חשיבה של 9.8 שניות נותנים 500 משתמשים. אותם 500 משתמשים בלי זמן חשיבה ייצרו 2,500 בקשות לשנייה, עומס שונה לגמרי. לכן "תריצי 500 יוזרים" היא דרישה שלא אומרת הרבה.

מודל פתוח מול מודל סגור, ולמה זה קריטי

מודל סגור - closed model

כל משתמש וירטואלי שולח בקשה, מחכה לתשובה ורק אז שולח את הבאה. זה ברירת המחדל ב-Thread Group של JMeter וב-constant-vus של k6.

מודל פתוח - open model

בקשות חדשות נכנסות בקצב קבוע, בלי קשר לשאלה אם הקודמות הסתיימו. ב-k6 זה constant-arrival-rate או rate, ובכלים אחרים זה wrk2 או Vegeta.

השמטה מתואמת - coordinated omission

המונח טבע Gil Tene, מייסד-שותף ו-CTO של Azul Systems. הוא תיאר את הבעיה ב-2013 ב-thread בשם "Coordinated Omission" בקבוצת mechanical-sympathy, ובהרצאה "How NOT to Measure Latency". במודל סגור, כשהמערכת נתקעת, גם מחולל העומס "נתקע איתה" ומפסיק לשלוח. הבקשות

שהיו אמורות לצאת בזמן התקיעה לא נמדדות בכלל. התוצאה: אחוזונים גבוהים שנראים מצוינים אף שהמשתמשים סבלו.

הדוגמה של Tene: מערכת עונה בתוך מילישנייה על 100 בקשות בשנייה, ואז נתקעת ל-100 שניות. כלי סגור יראה 10,000 תוצאות של מילישנייה ותוצאה אחת של 100 שניות, כלומר אחוזונים שנראים מצוינים. לפי השקפים של Tene (לונדון, 2013), מדידה נכונה, מהזמן שבו הבקשה הייתה אמורה לצאת, מראה ש-50p הוא מילישנייה, 75p הוא 50 שניות, 99.99p הוא כ-100 שניות, וזמן התגובה הממוצע הוא כ-25 שניות.

k6 אומרים את אותו הדבר בתיעוד שלהם: במודל סגור, "Slower response times means longer iterations and a lower arrival rate of new iterations".

איך מתגוננים:

- מודדים latency מהזמן המתוכנן לשליחה, לא מהזמן שבו הבקשה יצאה בפועל.
- משתמשים במודל פתוח כשהשאלה היא "האם המערכת עומדת בקצב X".
- רושמים latency ב-HdrHistogram, שיש בו גם תיקון מובנה ל-coordinated omission.

מתי כן משתמשים במודל סגור? כשבועולם האמיתי יש אוכלוסייה סגורה שמחכה לתשובות. דוגמאות: מספר קבוע של עמדות מפעיל, או מספר קבוע של יחידות קצה שכל אחת מחכה ל-ACK לפני שהיא שולחת את ההודעה הבאה. במערכות תקשורת ורדיו, שבהן מגיעים זרמי נתונים בקצב קבוע (פריימים, טלמטריה, הודעות), כמעט תמיד צריך מודל פתוח.

איך גוזרים יעד עומס

1. מתחילים מהדרישה או מה-SLO. מיקרוסופט: "Make sure these targets align with your service-level objectives (SLOs), and translate them into measurable test objectives". ההערה שלהם: לא לקבוע SLO לפני שמבינים את זרימות המשתמש (חלק 1, אסטרטגיה 1).
2. מנתחים דאטה מייצור, או מבצעים (spec) של המערכת במערכות חדשות: קצב ממוצע, קצב בשעת השיא, ותמהיל פעולות לפי ה-operational profile.
3. מתכננים לשיא ולא לממוצע, ומוסיפים מרווח ביטחון. ממוצע יומי מסתיר את שעת השיא.
4. בונים צורה בזמן: ramp-up, מצב יציב ארוך מספיק, ואז שלבי שיא וקפיצה.
5. מגדירים קריטריון מעבר/כישלון באחוזונים. הדוגמה של מיקרוסופט: אם ה-SLO הוא 95% מהבקשות תוך 200ms, הסף הוא $p_{95} \leq 200ms$, וכל ריצה מעל זה נכשלת.
6. מחלקים תקציב ביצועים בין השכבות. בדוגמה של מיקרוסופט: 400ms ל-API, 150ms ל-DB ועד 1% שגיאות. ככה, כשבדיקה נכשלת, יודעים איפה לחפש.

פרק 4: מה זה JMeter

אפאצ'י ג'יי-מיטר - Apache JMeter

לפי האתר הרשמי: "open source software, a 100% pure Java application designed to load test"

functional behavior and measure performance". הוא נבנה במקור לאפליקציות ווב והתרחב לפרוטוקולים נוספים.

רכיבי תוכנית בדיקה

תוכנית בדיקה - Test Plan

עץ של רכיבים שנשמר כקובץ JMX.

קבוצת תהליכונים - Thread Group

קובעת כמה "משתמשים" (threads) יהיו, מה זמן ה-ramp-up וכמה איטרציות. לפי התיעוד, כל thread מריץ את התוכנית "completely independently of other test threads". כל ה-samplers וה-controllers חייבים להיות בתוכה.

דוגם - Sampler

הרכיב ששולח את הבקשה בפועל, למשל TCP Sampler, JDBC Request, HTTP Request או JMS.

בקר לוגי - Logic Controller

שולט בזרימה: If, Loop, Transaction Controller ועוד.

טיימר - Timer

מוסיף השהיה לפני sampler. לפי התיעוד, בלי טיימר thread מריץ samplers ברצף "without pausing". כך מממשים think time.

בדיקת תקינות - Assertion

בודקת שהתשובה נכונה ולא רק מהירה. לפי התיעוד, שרת יכול להחזיר קוד הצלחה ועדיין להחזיר עמוד שגוי.

מאזין - Listener

אוסף את התוצאות ומציג אותן.

רכיבי תצורה ועיבוד - Config Elements, Pre/Post-Processors

משתנים, קובצי CSV, לפרמטרים, ו-extractors לקורלציה (JSON, XML, Regex).

מצב שורת פקודה והרצה מבוזרת

מצב ללא ממשק - CLI / non-GUI mode

כשפותחים את JMeter הוא מציג בעצמו את ההודעה "Don't use GUI mode for load testing". ה-GUI מיועד לבנייה ודיבאג בלבד. בדיקת אמת מריצים בשורת הפקודה, ואפשר לייצר בסוף דוח HTML מוכן:

```
jmeter -n -t plan.jmx -l results.jtl -e -o ./report
```

בדיקה מבוזרת - Remote/Distributed Testing

לקוח אחד שולט בכמה מכונות jmeter-server דרך RMI. לפי [המדריך הרשמי](#), לקוח JMeter יחיד על מעבד של 2-3GHz מחזיק 1000-2000 threads, תלוי בסוג הבדיקה. עוד מגבלה מהתיעוד: RMI לא עובר בין subnets בלי proxy.

- זה לא דפדפן. הוא לא מריץ JavaScript ולא מרנדר, ולכן לא מודד חוויית משתמש בצד הלקוח.
- ברירת המחדל היא מודל סגור, ולכן צריך להיזהר מ-coordinated omission. בשביל קצב קבוע יש timers כמו Constant Throughput Timer, או plugins.
- צריכת הזיכרון גבוהה. listeners כבדים כמו View Results Tree נועדו לדיבאג בלבד. לפי התייעוד, מצב "Functional Testing" מנפח את קובץ התוצאות ופוגע בביצועים.
- על Beanshell ו-JavaScript התייעוד ממליץ לוותר בעומס גבוה ולהעדיף Groovy, שמתקמפל.

פרק 5: האם JMeter מתאים לבדיקות חומרה?

התשובה הקצרה

JMeter לא בודק חומרה. הוא בודק ממשקי תוכנה שהחומרה חושפת. אם למכשיר שלך יש ממשק TCP, REST, MQTT, מסד נתונים או תור הודעות, JMeter או Locust יכולים להעמיס עליו ולמדוד זמני תגובה. מה שקורה בתוך המכשיר (CPU, זיכרון, תזמון דטרמיניסטי, bus פנימי) צריך למדוד בנפרד, עם כלים על המכשיר עצמו.

מה אפשר לעשות עם JMeter מול מכשירים:

- ל-TCP Sampler אפשר לשלוח הודעות בינאריות או טקסט לשרת TCP שרץ על המכשיר.
- ל-MQTT יש plugins של הקהילה. הם לא חלק מהליבה של JMeter, ולכן צריך לבדוק את הבשלות שלהם.
- JDBC ו-JMS מתאימים לבדיקת רכיבי backend במערכת, כמו תחנות קרקע ושרתי מידע.
- שימי לב שב-UDP, בפרוטוקולים קנייניים ובפרוטוקולים בזמן אמת JMeter לרוב לא מתאים. זה מוביל לכלים הבאים.

איך בודקים ביצועים של מערכות משובצות, תקשורת ורדיו

חומרה בלולאה - HIL (Hardware-in-the-Loop)

המכשיר האמיתי מחובר לסימולטור שמדמה את הסביבה: חיישנים, bus, רשת או ערוץ רדיו. מזריקים עומס ומודדים תגובה בזמן אמת. זו הדרך המקובלת לבדוק תזמון ודטרמיניזם.

רתמת בדיקה ייעודית - custom test harness

סקריפט Python או ++C/C שמדבר את הפרוטוקול של המכשיר (CAN, serial, socket), שולח בקצב מבוקר ומודד. זה המקום שבו Python שלך הכי שימושי. אפשר לכתוב רתמה כזו כ-custom client של Locust ולקבל ממנו הפצה ודוחות.

מחוללי תעבורת רשת - network traffic generators

- **iperf3**: כלי פשוט למדידת רוחב פס בין שתי נקודות. במצב UDP הוא מדווח גם jitter ואיבוד חבילות. מתאים לבדוק לינק, מודם או רדיו.

- **Cisco TRex**: מחולל תעבורה בקוד פתוח שמבוסס על DPDK, עם מצב stateless ומצב stateful. לפי ההכרזה של Andrew Harvey מ-Cisco ברשימת (2015) dpdk-dev, הוא "can scale to jitter-latency", ומספק סטטיסטיקה, ומספק סטטיסטיקה, ומספק סטטיסטיקה, ומספק סטטיסטיקה. לכל זרם, עם API ב-Python לאוטומציה.
- **Spirent-Keysight (Ixia)**: מכשירי בדיקה מסחריים ברמת line-rate. בדרך כלל משתמשים בהם לבדיקות של ציוד תקשורת לפי מתודולוגיות RFC 2544 ו-Y.1564.

עומס על CAN/serial bus - bus

במערכות רכב, תעופה ומשובצות מייצרים עומס על ה-bus עצמו, למשל עם python-can או can-utils. ואז בודקים שהודעות קריטיות עדיין מגיעות בזמן.

המדדים שחשובים בחומרה ובתקשורת

- **latency - השהיה**: לפי אחוזונים ו-max, לא לפי ממוצע. במערכות זמן אמת ה-max הוא לרוב הדרישה עצמה, כי דדליין שהוחמץ פעם אחת הוא כישלון.
- **jitter - ריצוד**: השונות בזמן ההגעה או העיבוד. קריטי בקול, וידאו, בקרה וסנכרון.
- **packet loss - איבוד חבילות**: כמה חבילות לא הגיעו, ובאיזה עומס זה מתחיל.
- **throughput - תפוקה**: קצב נתונים אפקטיבי מול הקצב הנומינלי של הלינק.
- **deterministic timing - דטרמיניזם של תזמון**: האם זמן העיבוד חסום ויציב גם בעומס.
- **on-device CPU/memory - משאבים על המכשיר**: ניצול, תורים ודליפות. כאן שיטת USE עובדת מצוין.

פרק 6: מתודולוגיה מקצועית של בדיקות ביצועים

יעדים, בסיס והשערות

- מגדירים יעדים מדידים לפי SLO ותקציבים לכל שכבה, כמו בפרק 3.
- קבעי baseline. מיקרוסופט: "Record performance metrics during initial tests. This recording is your baseline". כל ריצה חדשה משווים ל-baseline, וכך מזהים regression ו-drift.
- עבדי עם השערות. מיקרוסופט מציעה ניסויים מבוססי השערה, למשל "Adding an index to the orders table reduces query time by 70% under peak load". מריצים את אותה בדיקה על baseline ועל variant ומשווים.

סביבה, חימום ונתונים

- הסביבה צריכה להיות דומה לייצור. במערכות קריטיות מיקרוסופט דורשת התאמה מלאה של compute, autoscaling, caching, רשת ותלויות. ל-Rafael זה אומר: אותה חומרה, אותה גרסת firmware ואותו לינק.
- חימום. מתעלמים מהדקות הראשונות (JIT, caches, connection pools) ומודדים רק במצב יציב.
- נתונים ריאליים. מיקרוסופט ממליצה אפילו לא לעשות mock לתלויות חיצוניות בבדיקת עומס, כי mock מסתיר latency אמיתי.

- בודקים את מחולל העומס עצמו. אם ה-CPU של המחולל ב-100%, מה שמדדת הוא המחולל ולא המערכת.

אחוזונים, לא ממוצעים

- ב-SRE של Google מסבירים שמערכת שמחשבת אחוזונים (p50, p95, p99) מראה לך אם 50%, 5% או 1% מהבקשות איטיות מדי. ממוצע אומר רק שמהו "יותר איטי", בלי פירוט.
- תמיד מדווחים גם max, ומציגים את כל התפלגות האחוזונים. זו ההמלצה של Gil Tene.
- מובהקות סטטיסטית: מדידות ביצועים הן רועשות. מריצים כמה פעמים, בודקים שונות, ולא מסיקים מסקנות מריצה בודדת או מהפרש קטן. כדי שאחוזון גבוה כמו p99.9 יהיה אמין צריך הרבה דגימות.

ניטור בזמן הבדיקה: USE, RED ו-Golden Signals

שיטת USE - Utilization, Saturation, Errors

של Brendan Gregg: "For every resource, check utilization, saturation, and errors". לכל משאב (CPU, זיכרון, דיסק, כרטיס רשת, bus) בודקים:

- **ניצול:** אחוז הזמן שהמשאב עסוק.
- **רוויה:** אורך התור, כלומר עבודה שממתינה.
- **שגיאות:** ספירת אירועי שגיאה.

לפי Gregg, מעל 70% ניצול עיכובי תור הופכים תכופים ומורגשים, וכל רוויה שאינה אפס מוסיפה latency. הוא מזהיר גם שממוצע של 5 דקות יכול להסתיר דקות שלמות של 100%.

שיטת RED - Rate, Errors, Duration

הצד של השירות: קצב בקשות, שגיאות ומשך.

ארבעת האותות - Four Golden Signals

מתוך ספר ה-SRE של Google: latency, traffic, errors, saturation.

איך לחשוב על זה: RED ו-Golden Signals אומרים לך שיש בעיה. USE אומר לך איזה משאב הוא צוואר הבקבוק.

ניתוח צווארי בקבוק

1. מעלים עומס בהדרגה, ומחפשים את הברך: הנקודה שבה latency מתחיל לטפס והתפוקה מפסיקה לעלות.
2. באותה נקודה מריצים USE על כל המשאבים ומוצאים את המשאב הרווי.
3. מעמיקים עם profiling או flame graphs, מתקנים, ומריצים שוב מול ה-baseline.

Shift-left ו-CI/CD

- מיקרוסופט: להתחיל מוקדם, גם לפני שיש מערכת מלאה, עם profiling מקומי. אחר כך לשלב בדיקות ב-pipeline ולהריץ אותן באופן תקופתי כדי לתפוס drift.

- בפועל: smoke load קצר בכל PR עם thresholds שמכשילים את ה-build, ובדיקות ארוכות (soak, stress) בלילה או לפני שחרור.

טעויות נפוצות

- להגדיר עומס כ"מספר יוזרים" בלי קצב, בלי תמהיל ובלי think time.
- לדווח ממוצע בלי אחוזונים ובלי max.
- למדוד במודל סגור ולהסיק מזה על עמידה בקצב (coordinated omission).
- להריץ עומס מה-GUI של JMeter, או עם listeners כבדים.
- לא לנטר את מחולל העומס.
- בדיקה קצרה מדי, שלא מגלה דליפות. בשביל זה יש soak.
- סביבה שלא דומה לייצור, או mock לכל התלויות.
- לבדוק רק שהמערכת "מהירה" בלי assertions, כשבפועל היא מחזירה שגיאות מהר.

פרק 7: דוגמת קוד - מחולל עומס פתוח ב-Python שמודד latency

נכון

זה סקריפט פשוט שמעמיס בקצב קבוע על שרת TCP, למשל מכשיר שעונה ל-PING. הוא מודד latency מהזמן המתוכנן לשליחה, ולכן לא נופל ל-coordinated omission. הכתובת והפורט הם דוגמה.

```
"""Open-loop TCP latency probe that avoids coordinated omission."""

import asyncio
import statistics
import time

TARGET_HOST = "192.168.1.50"
TARGET_PORT = 5000
RATE_PER_SEC = 200
DURATION_SEC = 30
TIMEOUT_SEC = 1.0

async def send_one(
    intended_start: float,
    results: list[float],
    errors: list[str],
) -> None:
    # Latency is measured from the intended send time, not the actual one
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(TARGET_HOST, TARGET_PORT),
            TIMEOUT_SEC,
        )
        writer.write(b"PING\n")
        await writer.drain()
        await asyncio.wait_for(reader.readline(), TIMEOUT_SEC)
        writer.close()
        await writer.wait_closed()
        results.append((time.perf_counter() - intended_start) * 1000.0)
    except (OSError, asyncio.TimeoutError) as exc:
        errors.append(type(exc).__name__)

def report(results: list[float], errors: list[str], total: int) -> None:
```

```

if len(results) < 2:
    print(f"Not enough samples: ok={len(results)} errors={len(errors)}")
    return
cuts = statistics.quantiles(results, n=1000, method="inclusive")
print(f"sent={total} ok={len(results)} errors={len(errors)}")
print(f"mean={statistics.fmean(results):.2f} ms")
print(f"p50={cuts[499]:.2f} ms p95={cuts[949]:.2f} ms p99={cuts[989]:.2f} ms")
print(f"max={max(results):.2f} ms")

async def main() -> None:
    results: list[float] = []
    errors: list[str] = []
    tasks = []
    interval = 1.0 / RATE_PER_SEC
    total = RATE_PER_SEC * DURATION_SEC
    start = time.perf_counter()
    for i in range(total):
        intended = start + i * interval
        delay = intended - time.perf_counter()
        if delay > 0:
            await asyncio.sleep(delay)
        # Fire on schedule without waiting for previous responses (open model)
        tasks.append(asyncio.create_task(send_one(intended, results, errors)))
    await asyncio.gather(*tasks)
    report(results, errors, total)

if __name__ == "__main__":
    asyncio.run(main())

```

שימי לב: בסקריפט הזה אין הגבלה על מספר החיבורים הפתוחים במקביל. במכשיר חלש זה עלול להפוך בעצמו לבדיקת stress. אפשר להוסיף `asyncio.Semaphore`, אבל אז צריך לזכור שה-latency שנמדד כולל גם את ההמתנה לסמפור, וזה בכוונה.

המלצות

- ללמוד לפי הסדר הזה: קודם מושגים (מודל פתוח/סגור, אחוזונים, USE), אחר כך Locust ב-Python, ואז JMeter ברמה שמאפשרת לקרוא ולהריץ תוכניות קיימות.
- לפני כל בדיקה לכתוב במסמך קצר: מה המטרה (load, stress או soak), מה הקצב ומה התמהיל, מה ה-SLO באחוזונים, ומה מנטרים.
- במערכות חומרה ותקשורת להפריד בין שכבות. את הלינק בודקים עם iperf3 או TRex, את הפרוטוקול האפליקטיבי עם רתמת Python או Locust, ואת התזמון הפנימי עם HIL ומדידה על המכשיר.
- לנטר תמיד גם את המכשיר (USE) וגם את מחולל העומס.

הסתייגויות

- את הגדרות ה-ISTQB לקחתי מסילבוס CT-PT 2018 ומגרסאות קודמות של המילון. את הניסוח הנוכחי (x.4) באתר המילון לא הצלחתי לאמת ישירות, כי האתר לא נטען בלי JavaScript. יכול להיות שהניסוח השתנה מעט.

- חלק מהמידע על כלים מסחריים (LoadRunner, NeoLoad, BlazeMeter) ועל כלים כמו Gatling, Artillery, Vegeta ו-iperf3 מבוסס על ידע כללי בתחום ולא על תיעוד רשמי שבדקתי. כדאי לאמת יכולות ורישוי לפני שבחרים.
- את מספרי ה-scale של Azure (400 מנועים ו-100,000 משתמשים), של JMeter (1000-2000 threads) ושל TRex (200Gbps) מדווחים היצרנים עצמם, והם תלויים מאוד בתרחיש.
- ה-plugins של JMeter ל-MQTT הם של הקהילה ולא חלק מהליבה.

שני תיקונים לעומת הנוסח הקודם של הדוח: (1) ברשימת הסוגים של ISTQB, בדיקת הביצועים הבסיסית היא אחד משמונת הסוגים ולא תוספת להם, ולכן הרשימה מונה עכשיו שמונה סוגים. (2) הוסרה הטענה ש-Gatling "היה מהראשונים שהתייחסו ל-coordinated omission", כי היא לא אומתה מול מקור.

מקורות

- [Microsoft – Architecture strategies for performance testing \(PE:06\)](#)
- [Microsoft – Key concepts for Azure Load Testing](#)
- [Microsoft – What is Azure Load Testing](#)
- [ISTQB – Performance Testing Syllabus \(CT-PT\)](#)
- [ISTQB Glossary – Load Testing](#)
- [ISO/IEC 25010:2023 – sample](#)
- [Grafana k6 – Open and closed models](#)
- [Apache JMeter – Elements of a Test Plan](#)
- [Apache JMeter – Remote \(Distributed\) Testing](#)
- [Apache JMeter – Best Practices](#)
- [Locust – Testing other systems/protocols](#)
- [Gil Tene – Coordinated Omission](#)
- [High Scalability – Your Load Generator is Probably Lying to You](#)
- [Brendan Gregg – The USE Method](#)
- [Google SRE Workbook – Monitoring](#)
- [AWS – Distributed Load Testing on AWS](#)

חלק 3: איפה לראות פרויקט בדיקות ואוטומציה

מקצועי אמיתי

בשורה התחתונה: כדאי להתחיל מ-labgrid, מ-pytest-embedded ומ-OpenHTF. שלושתם פרויקטי Python פעילים שמראים איך בנויה תשתית בדיקות מקצועית לחומרה: שכבת דרייברים, fixtures של pytest, קונפיגורציה לכל סביבה ו-CI. אחריהם כדאי לעבור ל-pyATS ול-Scapy בשביל רשתות ופרוטוקולים, ובסוף לקרוא את תיקיות הבדיקות של pytest ו-httpx כדי לראות איך נראה "קוד בדיקות נקי".

3.1 תקציר

- שלושת הרפוזיטוריוז החשובים ביותר ללמידה הם **labgrid** (529 Pengutronix, כוכבים, גרסה v26.0 ביוני 2026), **pytest-embedded** (Espressif, גרסה v2.9.3 מ-31 באוגוסט 2026) ו-**OpenHTF** (723 Google, כוכבים). כל אחד מהם מראה בצורה מפורשת הפרדה בין הבדיקה לבין הדרייבר של המכשיר.
- לבדיקות של תקשורת, רשתות ופרוטוקולים: **pyATS/Genie** של Cisco (בדיקות מערכת על טופולוגיה שמוגדרת ב-YAML), **Scapy** (בניית חבילות ובדיקות רגרסיה של פרוטוקולים), ו-**TREx** יחד עם **snappi** (מחוללי תעבורה שמופעלים דרך API ב-Python).
- כדאי לקרוא את הקוד בסדר קבוע: קודם `examples/`, אחר כך `conftest.py`, אחר כך שכבת הדרייברים, ובסוף `github/workflows`. חשוב להבין את השכבות (בדיקה ← לוגיקה/keywords ← דרייבר ← חומרה) לפני שנכנסים לפרטים.

3.2 תשתיות לבדיקות חומרה, embedded ומערכת

labgrid

ספריית שליטה על לוחות - labgrid

ספריית Python לשליטה על לוחות embedded, כלומר על ספק כוח, קונסולה סריאלית, SSH ושינוי מצבי אתחול. יש לה פלאגין ל-pytest ושכבת exporter-coordinator שמאפשרת להשתמש בלוחות מרוחקים ברשת. Pengutronix התחילה לפתח אותה ב-2016, והיא עדיין בפיתוח פעיל.

הכי חשוב ללמוד ממנה: היא "לא מסגרת בדיקות". היא שכבת הפשטה של החומרה שמשתלבת עם pytest, וזה בדיוק המבנה שתצרי לבנות בעבודה.

כדי לראות שהפרויקט אמין: יש לו 529 כוכבים ו-3,689 קומיטים, הגרסה האחרונה היא v26.0 מיוני 2026, וגם Analog Devices מתחזקת fork משלה.

מאיפה להתחיל: `tests/`, `examples/` ו-`github/` (שם יש workflows של unit tests ו-docker build).

מושגי הליבה ב-labgrid:

מושב	משמעות
Target	הלוח עצמו, עם כל המשאבים והדרייברים שלו
Resource	משאב פיזי, למשל פורט סריאלי או שקע חשמל
Driver	הקוד שמדבר עם המשאב
Strategy	מעבירה את הלוח בין מצבים, למשל "כבוי" ← "Linux shell" ← "U-Boot"

ה-Strategy היא תבנית העיצוב State, שממושת כאן בעולם אמיתי.

pytest-embedded

פלאגין לבדיקות embedded - pytest-embedded

פלאגין של Espressif ל-pytest שמיועד לבדיקות embedded. הוא בנוי כ"שירותים" נפרדים (serial, idf), דאקו (qemu, wokwi, arduino, nuttx), ומפעילים אותם בעזרת `embedded-services`. הבדיקה עצמה נראית כך: `dut.expect("...")`, כלומר מחכים לפלט מהמכשיר.

מצב הפרויקט: הגרסה האחרונה היא v2.9.3 מ-31 באוגוסט 2026, ויש לפרויקט 153 כוכבים.

למה ללמוד ממנו: רואים איך מפרקים תשתית גדולה לחבילות קטנות (pytest-embedded-serial, pytest-embedded-qemu, pytest-embedded-idf), ואיך אותה בדיקה רצה על חומרה אמיתית או על אמולטור.

מאיפה להתחיל: `examples/esp-idf/hello_world/test_hello_world.py`, קובץ `conftest.py` שבשורש, ותיקיות החבילות.

OpenHTF

מסגרת לבדיקות חומרה - OpenHTE

מסגרת לבדיקות חומרה שמקורה ב-Google (בעמוד שלה כתוב "not an official Google product"). אפשר להשתמש בה מהמעבדה ועד רצפת הייצור.

מצב הפרויקט: 723 כוכבים, 238 forks ו-111 issues פתוחים. היא לא מתפתחת מהר, אבל עדיין מתוחזקת.

ארבעת המושגים שהיא בנויה סביבם:

מושב	משמעות
Test	הבדיקה כולה
Phase	שלב בבדיקה
Measurement	ערך שנמדד מול גבולות
Plug	עטיפה למכשיר מדידה או ל-DUT

תקע - Plug

המקבילה של "דרייבר" ב-OpenHTF: מחלקה שעוטפת מכשיר מדידה או את ה-DUT. יש לה מחזור חיים מנוהל, כלומר `tearDown()` נקרא אוטומטית בסוף הבדיקה.

התקן הנבדק - DUT (Device Under Test)

הכרטיס, הרכיב או המערכת שבודקים.

מדידה עם גבולות - Measurement

ערך שנמדד ונבדק מול גבולות, למשל מתח בין 3.2 ל-3.4 וולט. התוצאה נשמרת ברשומה מסודרת. זה מאוד רלוונטי לבדיקות ATP ולבדיקות קבלה.

מאיפה להתחיל: `examples/all_the_things.py` (קובץ אחד שמדגים את כל היכולות), `/openhft/util/test.py` (איך בודקים phases עם plugs מדומים) ו-`openhft/core/test_state.py`.

LAVA

מעבדה כשירות - LAVA

מערכת CI של Linaro שמתקינה מערכות הפעלה על חומרה פיזית ווירטואלית ומריצה עליהן בדיקות. משתמשים בה גם כדי לשתף לוחות בין צוותים. היא מיועדת בעיקר ללינוקס על ARM. יש לה גם [מראה ב-GitHub](#).

מה ללמוד ממנה: ארכיטקטורה של "מעבדה כשירות" (server ו-dispatcher), ומבנה של job ב-YAML. היא כבדה מדי כנקודת התחלה, אבל מלמדת איך נראית מעבדה בקנה מידה גדול.

tbot

אוטומציה ל-Linux embedded - tbot

כלי לאוטומציה ובדיקות של embedded Linux. הוא מנהל כמה חיבורים במקביל: serial ללוח, SSH לשרת המעבדה, TFTP ועוד.

מצב הפרויקט: כ-100 כוכבים ו-1,476 קומיטים. פקודת ההתקנה ב-README מצמידה גרסה v0.10.9 או v0.10.10 (עותקים שונים של העמוד מציגים גרסאות שונות, אז כדאי לבדוק את ה-tags בריפו).

דוגמה טובה: `test_ethernet_speed`. מריצים שרת iperf על מחשב המעבדה ולקוח על ה-DUT, וזה תרחיש בדיקה של תקשורת בין שני צדדים.

מאיפה להתחיל: `tbot_contrib`, `/selftest/` וה-workflow בשם "tbot selftest CI".

Avocado

מסגרת בדיקות של Red Hat - Avocado

מסגרת בדיקות שיצאה מ-Autotest. בדיקות Native נכתבות בסגנון unittest, אבל כל קובץ הרצה יכול לשמש כבדיקה. היא מוציאה דוחות TAP, JSON, xUnit ו-HTML, ויש לה Job Replay ו-Job Diff.

מצב הפרויקט: הגרסה האחרונה היא 113.0 מינואר 2026, ויש לה 368 כוכבים.

מה ללמוד ממנה: את (process.py, linux.py, kernel.py) avocado/utils/ כדאי לקרוא כספריית עזר לבדיקות מערכת, למשל הרצת תהליכים וקריאת /proc/sys/. כדאי להתחיל גם מ-examples/ ומ-selftests/.

Robot Framework

מסגרת אוטומציה מבוססת keywords - Robot Framework

מסגרת אוטומציה גנרית שמבוססת keywords. מאחוריה עומדת Robot Framework Foundation, ארגון ללא מטרות רווח.

מצב הפרויקט: 11.9 אלף כוכבים, והגרסה היציבה האחרונה היא 7.4.2 ממרץ 2026.

לעולם החומרה: רלוונטיות במיוחד SerialLibrary ו-SSHLibrary. חברת DornerWorks פרסמה מדריך HIL שבו suite בשם boot.robot משתמש ב-SerialLibrary, ובדיקות ההמשך מתחברות ללוח דרך SSHLibrary.

הרשימה המרכזית של האקוסיסטם: [awesome-robotframework](https://awesome-robotframework.github.io/).

בריפו הראשי כדאי להסתכל על: atest / (בדיקות קבלה שכתובות ב-Robot עצמו) ו-utest.

Renode

סימולטור למערכות embedded - Renode

סימולטור של Antmicro למערכות embedded מרובות צמתים, עם תקשורת קווית ואלחוטית. כותבים בו בדיקות ב-Robot Framework.

יש גם [GitHub Action מוכן](https://github.com/renode/robot) שמריץ בדיקות robot ב-Renode בתוך CI.

מה ללמוד ממנו: איך בודקים קושחה ברשת של כמה מכשירים בלי חומרה, ואיך מכניסים את זה ל-CI.

cocotb

אימות חומרה ב-Python - cocotb

מסגרת לכתיבת testbenches ב-Python ל-VHDL ול-Verilog, כלומר אימות RTL מבוסס קורוטונים.

מצב הפרויקט: 2,514 כוכבים, והיא עודכנה בספטמבר 2026.

מתי רלוונטי: אם תבדקי FPGA או לוגיקה דיגיטלית. אם לא, מספיק לדעת שהיא קיימת.

מריץ הבדיקות של Zephyr - Twister

מריץ הבדיקות של Zephyr RTOS. הוא מגלה בדיקות, בונה אותן לכמה פלטפורמות ומריץ אותן דרך harnesses (ztest, console, pytest, robot, power). יש לו "hardware map" ב-YAML שמתאר אילו לוחות מחוברים ולאיזה פורט.

הפלאגין pytest-twister-harness מאפשר לכתוב בדיקות pytest מול DUT אמיתי, ויש דוגמה ב-samples/pytest/subsys/testsuite/ בריפו של Zephyr.

מה כדאי להעתיק: את רעיון ה-hardware map לפרויקטים שלך.

KernelCI

בדיקת ליבת לינוקס במעבדות מבוזרות - KernelCI

פרויקט קהילתי לבדיקת ליבת לינוקס על חומרה אמיתית במעבדות מבוזרות. יש לו מתזמן בשם Maestro, מסד תוצאות בשם KCIDB, ומעבדות מבוססות LAVA או מעבדות "pull".

מעבדת pull: לפי הבלוג של KernelCI Foundation, מעבדה כזו יכולה לשבת מאחורי firewall, מצב נפוץ מאוד בסביבות ארגוניות. runtime מבוסס pull נוסף ברבעון האחרון של 2025.

מי משתמש: לפי עדכון של KernelCI Foundation מפברואר 2025, Pengutronix חיברה מעבדה משלה ל-Maestro, MediaTek הוסיפה חומרה דרך המעבדה של Collabora, ו-Qualcomm עבדה אז על חיבור מעבדות נוספות. בהמשך דווח ש-RISC-V International, Qualcomm, ו-Texas Instruments חיברו מערכות בדיקה.

מה ללמוד ממנו: ארכיטקטורה של מעבדות מבוזרות. לא צריך לקרוא את הקוד לעומק.

3.3 רשתות, פרוטוקולים ותעבורה

Genie ו-pyATS

מערכת הבדיקות של Cisco - pyATS

לפי עמוד pyATS ב-Cisco DevNet, היא פותחה לשימוש פנימי של Cisco מ-2014, היא הפתרון המקובל לבדיקות ואימות שבו משתמשים אלפי מהנדסים של Cisco, והיא זכתה ב-Cisco Pioneer Award ב-2018.

משתמשים בה לבדיקות scale ו-sanity, feature, solution, system על נתבים, מתגים, נקודות גישה ו-firewalls. הטופולוגיה מוגדרת בקובץ testbed ב-YAML, ו-Genie מפרסר פלט של מכשירים למבני נתונים.

שימי לב: הליבה של pyATS סגורה ומופצת כבינארי. [הריפו pyats](#) מכיל רק issue tracker. את הקוד הפתוח ששווה לקרוא תמצאי ב-[CiscoTestAutomation/examples](#) (תיקיות basic, comprehensive /libraries/ עם מכשירים מדומים), וב-genielibs וב-genieparser.

הדבר העיקרי ללמוד: הפרדה בין "מה בודקים" (testscript) לבין "על מה בודקים" (testbed YAML). זה רלוונטי ישירות למערכות תקשורת מרובות רכיבים.

Scapy

בניית חבילות רשת - Scapy

ספרייה לבנייה, שליחה, לכידה ופענוח של חבילות רשת בעשרות פרוטוקולים.

מצב הפרויקט: 12.4 אלף כוכבים, והגרסה האחרונה היא v2.7.0 מדצמבר 2025.

מריץ בדיקות משלה: קוראים לו UTscapy. בו test campaign מתחלק ל-test sets, ואלה מתחלקים ל-unit tests עם תגיות.

מה לקרוא:

- test/regression.uts
- test/contrib/automotive/uds.uts (בדיקת פרוטוקול תעשייתי)
- github/workflows/unittests.yml, שמראה matrix של גרסאות Python ומצבי root ו-non-root

TRex

מחולל תעבורה - TRex

מחולל תעבורה stateful ו-stateless של Cisco שמבוסס DPDK. לפי המדריך של TRex באתר של Cisco, הוא מגיע עד 200Gb/s על שרת UCS אחד (עם כרטיסי רשת של Intel ב-40Gb/s). יש לו API ב-Python שעובד ב-JSON-RPC2 מעל ZMQ.

מאיפה להתחיל: doc/trex_cookbook.asciidoc. שם רואים איך בונים חבילה ב-Scapy, מזריקים אותה דרך STLClient ומשחררים את החיבור ב-finally.

snappi / Open Traffic Generator

ממשק אחיד למחוללי תעבורה - snappi

SDK ב-Python וב-Go לתקן פתוח ובלתי תלוי ביצרן של מחוללי תעבורה בשכבות 2 עד 7. אותו סקריפט רץ על Ixia-c, על LxNetwork של Keysight ועל TRex (דרך snappi-trex).

מצב הפרויקט: הגרסה האחרונה היא v1.61.1 מאוגוסט. הדוגמה ב-README היא בדיקת pytest מלאה עם @pytest.mark.example. דוגמאות מוכנות נוספות יש ב-[otg-examples](https://github.com/ixia-c/otg-examples).

מה ללמוד ממנו: זו דוגמה מצוינת לתבנית Adapter: API אחד, ומימושים שונים לצידוד שונה.

3.4 פרויקטים עם תיקיית בדיקות לדוגמה

pytest עצמו

תיקיית testing / ב-pytest

14.5 אלף כוכבים, גרסה 9.1.1 מיוני 2026. שם רואים איך בודקים כלי בדיקות בעזרת ה-fixture בשם pytester, ואיך בנוי conftest.py גדול.

httpx

תיקיית tests / ב-httpx

15.5 אלף כוכבים. יש בה חלוקה ברורה לפי תחומים: client/, models / ו-fixtures/. יש גם tests / conftest.py שמרים שרת בדיקה, ו-test_whatwg.py שמריץ בדיקות מונחות-נתונים מתוך קובץ JSON.

Home Assistant

הריפו home-assistant/core

מחזיק אלפי אינטגרציות עם מכשירים. כל אינטגרציה מקבלת תיקייה ב-name- /tests/components/ עם conftest.py משלה וקבצי test_*.py, וההגדרה של setup ו-unload נמצאת ב-test_init.py. המבנה [מתועד רשמית](#).

זו הדוגמה הכי טובה שמצאתי לבדיקות של מכשירים מדומים בקנה מידה גדול.

הערה על CPython, Playwright ו-Kubernetes e2e

אלה פרויקטים מוכרים עם תשתית בדיקות רצינית, אבל לא בדקתי אותם לעומק במחקר הזה. הם גם פחות רלוונטיים לעולם החומרה. כדאי להגיע אליהם רק אחרי הרשימה שלמעלה.

3.5 תבניות ורשימות

הדמו הרשמי של Robot Framework - RobotDemo

מדגים שלוש גישות על אותו מחשבון: keyword-driven, data-driven ו-Gherkin. יש גם [WebDemo](#) רשמי.

רשימת כלים מנוהלת - awesome-test-automation

רשימה של כלים לכל שפה, עם 7,169 כוכבים. הקובץ python-test-automation.md הוא החלק הרלוונטי. הרשימה נותנת תמונה רחבה, אבל חלק מהקישורים בה ישנים.

הערה על pytest-harvest ו-"Lona"

את pytest-harvest לא אימתתי במחקר הזה. לא מצאתי פרויקט בדיקות מוכר בשם "Lona", ונראה שזה שם שגוי.

3.6 איך נראה פרויקט אוטומציה מקצועי

ארבע השכבות

כמעט כל הפרויקטים שלמעלה מתכנסים לאותו מבנה של ארבע שכבות:

שכבה 1 - מקרי בדיקה (test cases): קצרים וקריאים, בלי פרטי חומרה.

שכבה 2 - לוגיקת בדיקה או keywords: פעולות ברמת הדומיין, למשל "העבר את הרדיו לערוץ X". ב-Robot זה resource file, ובתבנית של labgrid זה Strategy.

שכבה 3 - דרייברים או HAL: מחלקה אחת לכל מכשיר או פרוטוקול. ב-OpenHTF זה Plug, ב-labgrid זה Driver, ובעולם התעבורה זה snappi.

שכבה 4 - חומרה ומשאבים: פורטים, כתובות IP וספקי כוח, שמוגדרים בקובץ קונפיגורציה ולא בקוד.

ההיגיון מאחורי זה: כשמחליפים מכשיר, משנים רק את שכבה 3. כשמחליפים מעבדה, משנים רק את הקונפיגורציה.

מבנה תיקיות טיפוסי

המבנה הבא מרכז את מה שחוזר ב-labgrid, ב-pytest-embedded, ב-Home Assistant וב-Zephyr:

```
my-system-tests/
├── pyproject.toml           # deps, pytest config, ruff config, markers
├── src/mylab/
│   ├── drivers/            # one class per device / protocol (HAL)
│   ├── keywords/          # domain-level actions built on drivers
│   └── utils/              # logging, retries, parsers
├── config/
│   ├── lab_a.yaml         # environment: ports, IPs, device map
│   └── sim.yaml           # same tests, simulated devices
├── tests/
│   ├── conftest.py        # shared fixtures (session-scoped devices)
│   ├── unit/              # tests for drivers with mocks, no hardware
│   ├── integration/       # one real device
│   └── system/            # full setup, marked as slow/hardware
├── test_data/              # vectors, pcaps, expected outputs
├── reports/                # JUnit XML / HTML / Allure output (gitignored)
└── .github/workflows/ci.yml
```

דוגמת קוד: fixture של מכשיר ובדיקה עם פרמטרים

הדוגמה מדגימה דרייבר נפרד מהבדיקה, fixture ברמת session עם teardown מובטח, marker parametrized-ruff. היא עוברת ruff.

```
"""Example: device fixture with a HAL-style driver and parametrized test."""
from collections.abc import Iterator
from dataclasses import dataclass
```

```

import pytest

@dataclass
class RadioDriver:
    """Thin driver layer: talks to the device, knows nothing about tests."""

    port: str
    connected: bool = False

    def connect(self) -> None:
        # Real code would open a serial/socket connection here
        self.connected = True

    def set_channel(self, channel: int) -> int:
        if not self.connected:
            raise RuntimeError("device not connected")
        # Real code would send a command and parse the reply
        return channel

    def close(self) -> None:
        self.connected = False

@pytest.fixture(scope="session")
def radio() -> Iterator[RadioDriver]:
    # Setup once per session, teardown always runs after yield
    driver = RadioDriver(port="/dev/ttyUSB0")
    driver.connect()
    yield driver
    driver.close()

@pytest.mark.hardware
@pytest.mark.parametrize("channel", [1, 6, 11])
def test_set_channel(radio: RadioDriver, channel: int) -> None:
    assert radio.set_channel(channel) == channel

```

חשוב לזכור: צריך לרשום את ה-marker בשם hardware ב-pyproject.toml, תחת `markers [tool.pytest.ini_options]`. אחרת pytest יוציא אזהרה. אחרי שהוא רשום, אפשר להריץ בלי חומרה עם `-m "not hardware"`.

כלים ומוסכמות שחוזרים בכל הפרויקטים

קיבוע - fixture

מקום אחד שבו המכשיר מוקם ומשוחזר. ב-pytest-embedded ה-fixture בשם dut הוא הלב של כל בדיקה.

סימון - marker

תגית שמסננת בדיקות: slow, hardware, או שם של לוח מסוים. אותו רעיון קיים גם ב-UTscapy בעזרת `~keyword`.

פרמטריזציה - parametrize

מריצים את אותה בדיקה על כמה ערכים, כמו ערוצים, קצבים או פרוטוקולים. ב-Robot זה נקרא Test Template, כלומר data-driven.

תיעוד ריצה - logging

שומרים כל פלט סריאלי וכל פקודה, עם חותמת זמן. pytest-embedded עושה את זה אוטומטית, ו-Avocado שומר job.log לכל ריצה.

דוחות - reporting

JUnit XML (בעזרת `--junitxml`) הוא השפה המשותפת לכל מערכות ה-CI, כמו Jenkins ו-GitHub Actions. ל-Avocado יש דוחות JSON, xUnit, Robot ו-HTML. מייצר report.html ו-Allure.log.html. תוסף נפוץ שמוסיף דוחות ויזואליים.

אינטגרציה רציפה - CI

כדאי להסתכל על `github/workflows/unittests.yml` של Scapy (matrix של גרסאות), על ה-workflow בשם "tbot selftest CI" ועל `renode-test-action`, כדי להבין איך מריצים בדיקות חומרה בסימולציה בכל `commit`. בדיקות על חומרה אמיתית רצות בדרך כלל על runner שמחובר למעבדה, וזה מה ש-LAVA ו-KernelCI עושים.

איכות קוד - linting ו-typing

ruff ל-lint ולפורמט, mypy ל-type hints, ו-pre-commit כדי שהבדיקות ירוצו אוטומטית לפני `commit`. ל-tbot יש קובץ `pre-commit-config.yaml`, ו-OpenHTF כבר עברה ל-type hints ומריצה בודק טיפוסים.

3.7 סדר קריאה ל-4 עד 6 שבועות

שלב 1 (שבוע 1) - pytest-embedded

הכי קרוב למה שאת כבר מכירה, כי זה pytest רגיל. קראי את `examples/esp-idf/hello_world`, אחר כך את `conftest.py` שבשורש, ואז את `pytest-embedded-serial` כדי לראות איך דרייבר סריאלי עוטף את `expect`.

תרגיל: כתבי דרייבר מדומה בסגנון הזה למכשיר דמיוני.

שלב 2 (שבועות 1-2) - labgrid

כאן תלמדי ארכיטקטורה: Resource, Driver, Strategy, וקונפיגורציה ב-YAML. קראי את `examples/`, ואז בחרי דרייבר אחד, למשל serial או power, ועקבי אחריו עד הבדיקה. כדאי גם לראות את סדרת ההדרכה של Pengutronix ביוטיוב שמוזכרת ב-README.

שלב 3 (שבוע 3) - OpenHTF

זה המודל של בדיקות קבלה וייצור: Phases, Measurements עם גבולות, ו-Plugs. קראי את `examples/all_the_things.py` ואז את `openhft/util/test.py` כדי לראות איך בודקים phases עם plugs מדומים. זה רלוונטי במיוחד לבדיקות ATP של כרטיסים.

שלב 4 (שבוע 4) - pyATS examples, Scapy ו-TRex

זה החלק של התקשורת. ב-CiscoTestAutomation/examples קראי את `basic/` ואת `libraries/` כדי להבין את ההפרדה בין testbed לבין script. ב-Scapy קראי קובץ `uts`. אחד ואת ה-workflow. ב-TRex קראי את ה-cookbook.

שלב 5 (שבועות 5-6) - קריאה של קוד "נקי"

קראי את `/tests` של `httpx` ואת `/testing` של `pytest` כדי ללמוד סגנון, ואת `/tests/components` של `Home Assistant` כדי לראות בדיקות של מכשירים מדומים בקנה מידה גדול.

המלצה מעשית לעבודה: לפני שאת כותבת שורת קוד, ציירי את ארבע השכבות של המערכת שלך. אחר כך בני את הפרויקט כך שכל הבדיקות ירוצו גם על `config/sim.yaml`, כלומר עם דרייברים מדומים. זה מה שמאפשר CI בלי מעבדה, וזה ההבדל הכי גדול בין פרויקט חובבני לפרויקט מקצועי.

3.8 הסתייגויות

- מספרי הכוכבים והתאריכים נלקחו מעותקים של עמודי GitHub מיולי עד ספטמבר 2026, ולכן ייתכנו סטיות קטנות. לא הצלחתי לאמת תאריך של `commit` אחרון, והתאריכים שמופיעים כאן הם תאריכי `releases`.
 - הליבה של `pyATS` סגורה. אפשר ללמוד מהשימוש בה ומהספריות הפתוחות, אבל לא מהקוד הפנימי שלה.
 - `LAVA` ו-`KernelCI` הן מערכות בקנה מידה של ארגון. לומדים מהן ארכיטקטורה, אבל הן לא נקודת התחלה טובה לקריאת קוד.
 - לפני שמכניסים כלי כלשהו לסביבה מסווגת, צריך לבדוק את מדיניות הרישוי והאבטחה של הארגון.
- `Scapy` ו-`Avocado` הם `GPLv2`, ו-`Robot Framework`, `labgrid` ו-`Renode` הם ברישיונות מתירניים יותר.